

EVERCARE

System Documentation

Database Design, Schema Overview, System Flow, and Architecture

Prepared by: Brent Lawrence C. Bernardo

Section: ITE231

Professor: Paul John Cabance

Document date: August 28, 2026

System: EverCare Flutter Mobile Application

Table of Contents

Table of Contents	1
Document Purpose and Scope	2
Executive Summary	2
System Overview	2
Users and Navigation	3
System Architecture	4
Core System Flow	6
Major Feature Workflows	8
Blood-Pressure Capture	8
Medication Adherence	8
Appointment Attendance	8
Journals and Private Photos	8
Emergency and Hospital Finder	8
Care Book, AI, Notifications, and Trusted People	8
Database Design Overview	9
Database Schema Dictionary	12
Identity and Trusted-Care Tables	12
Health and Scheduling Tables	12
Journal and Safety Tables	13
Database Security and Integrity	14
Row Level Security and Grants	14
Private Photo Storage	14
Server-Side Functions and Triggers	14
Implementation Reference	15
Current Scope and Limitations	15
Source of Truth	16
Conclusion	16

Document Purpose and Scope

This document describes the current EverCare mobile application as implemented in the repository snapshot dated August 28, 2026. It explains the system architecture, core application flow, PostgreSQL data model, Supabase security boundary, principal business rules, and the source-code structure that supports the application.

The Supabase migration files are treated as the authoritative database definition. Flutter source files are treated as the authoritative description of implemented user flows. The diagrams are intentionally rendered at print resolution on dedicated landscape pages so labels remain readable in the PDF and on paper.

Diagram convention: Solid boxes are implemented EverCare or Supabase components. Dashed boxes represent external or device-provided services. Arrows show the direction of a request, event, or persisted relationship.

Database convention: PK means primary key, FK means foreign key, RLS means Row Level Security, RPC means a PostgreSQL function called through Supabase, and a question mark after a type denotes a nullable column in the compact schema listing.

Executive Summary

EverCare is a Flutter care companion for Filipino seniors, caregivers, and family members. It brings blood-pressure records, medication schedules, medical appointments, journals with private photos, emergency details, caregiver relationship records, notifications, and caregiving references into one accessible mobile interface.

The mobile client connects to Supabase for email/password authentication, PostgreSQL persistence, PostgREST data access, server-validated RPCs, private object storage, and authenticated Edge Functions. It also integrates with a YK-IBPA1 Bluetooth Low Energy blood-pressure monitor, OpenStreetMap-based hospital services, external Google Maps links, the bundled NIA Caregiver's Handbook, and a Groq-hosted language model reached only through server-side Edge Functions.

Safety boundary: EverCare is not a diagnostic device, medical verification service, emergency-dispatch system, or substitute for professional care. Saved blood-pressure records are constrained to remain medically unverified. Emergency numbers are copied for the user; the app does not automatically call, text, send an SOS, or alert another person.

System Overview

Area	Implemented technology and responsibility
Mobile client	Flutter / Dart application with named routes, reusable widgets, an eight-tab MainShell, and elderly-friendly accessibility controls.
State	Screen-local StatefulWidget/FutureBuilder state plus app-owned ChangeNotifier scopes for BLE monitoring and accessibility settings.
Backend	Supabase Auth, PostgREST, PostgreSQL RPCs, Row Level Security, private Storage, and authenticated Edge Functions.

Area	Implemented technology and responsibility
Database	PostgreSQL 17 with 11 public application tables plus Supabase-managed auth.users and storage objects.
Hardware	YK-IBPA1 BLE monitor; the app decodes progress and final packets and applies the implemented local systolic calibration.
External services	OpenStreetMap tiles, Overpass, Nominatim, Photon, external Google Maps URLs, Groq API, and NIA reference resources.
Local device storage	SharedPreferences for text-size/reduced-motion settings and saved BLE monitor setup; camera/gallery, location, clipboard, files, and browser are used only when the related feature is invoked.

Users and Navigation

Registration accepts three profile values: senior, caregiver, and family member. These are descriptive profile metadata in the current release, not separate authorization roles. Every authenticated account receives the same main shell, and no administrator or clinician module exists.

- Home — dashboard summary, next medication, next appointment, and quick actions.
- My Health — BLE/manual blood-pressure capture, history, trend, and optional AI explanation or chat.
- Medications — weekday schedule, due/taken/missed state, course completion, edit, and delete.
- Appointments — create, edit, hospital selection, attendance state, directions, completion, cancellation, and delete.
- Journals — create, read, edit, bookmark, delete, and attach private photos.
- Care Book — bundled caregiving chapters, handbook download, references, and memory-only AI chat.
- Emergency — hotline/contacts, medical information links, safety guidance, and hospital finder.
- Profile — personal details, trusted people, accessibility, settings, notifications, and logout.

System Architecture

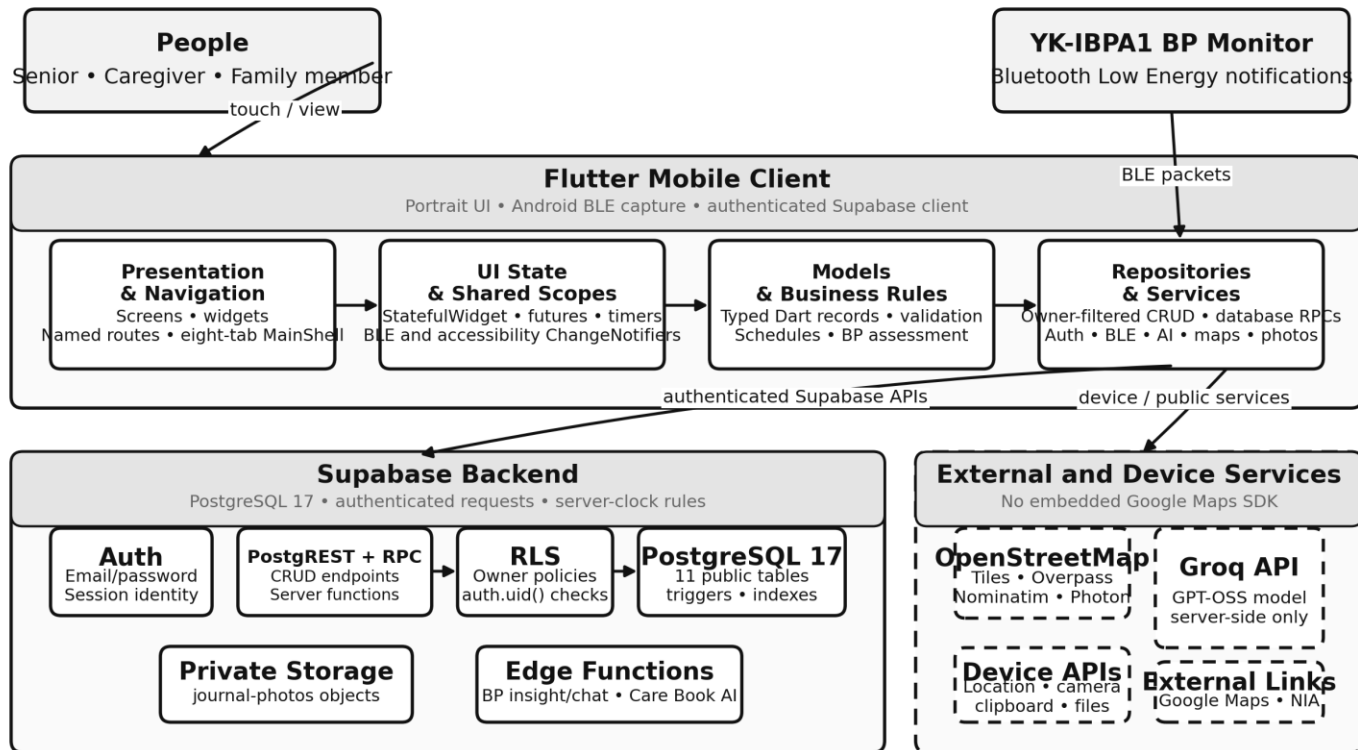
EverCare is layered, but it is not a strict Clean Architecture implementation. Screens commonly instantiate feature repositories directly. Shared dependencies are injected through Flutter inherited scopes, while repositories translate Supabase responses into typed Dart models.

Layer	Current implementation
Bootstrap and composition	Initializes Flutter, locks portrait orientation, initializes Supabase, creates BLE/accessibility controllers, and injects application scopes.
Presentation and navigation	Feature screens, widgets, named routes, route observers, headers, and the eight-tab MainShell.
State and application behavior	Screen-local state, futures, timers, lifecycle observers, and ChangeNotifier scopes for shared BLE and accessibility state.
Domain models and rules	Typed medication, dose, appointment, journal, BP, emergency, caregiver, hospital, profile, and notification records plus validation/schedule logic.
Repositories and integration services	Owner-filtered CRUD, RPC calls, Storage operations, Auth, BLE, AI, hospital lookup, photo picking, downloads, and local preferences.
Backend and external services	Supabase authentication/data/security/storage/functions; Groq, OpenStreetMap services, Google Maps URLs, and NIA references.

All direct database operations use the authenticated Supabase client and are still checked by server-side RLS. Client-side owner filters improve clarity and query efficiency but are not the security boundary.

EverCare System Architecture

Implemented Flutter client, Supabase backend, device services, and external integrations



Persistent health and care data is protected by Supabase authentication and Row Level Security; local preferences store only accessibility and BLE setup choices.

Figure 1. EverCare system architecture and component interaction.

Core System Flow

EverCare begins by initializing Supabase and the shared device/accessibility controllers. The splash screen checks the current Supabase session. A signed-in user enters the MainShell; otherwise, the app presents welcome, onboarding, login, registration, and password-reset paths.

1. Initialize Flutter bindings, system UI, Supabase, BLE monitoring state, and accessibility preferences.
2. Check the Supabase Auth session after the splash delay.
3. When no session exists, complete onboarding and authenticate or register. Registration metadata is copied into a new profiles row by a database trigger.
4. Enter the authenticated MainShell and choose one of the eight feature tabs or a detail route.
5. The selected screen applies local validation and calls a repository or integration service.
6. Supabase requests pass through authentication, grants, RLS, database constraints, triggers, and RPC rules before a response returns to the UI.
7. The screen refreshes, returns to the dashboard, or continues to another feature. In-app medication and appointment reconciliation runs again on the relevant load/resume paths.

Important distinction: Direct device or public-service actions—BLE, location, camera/gallery, clipboard, files, map tiles, and public hospital lookup—do not use the PostgreSQL path. AI calls do use an authenticated Edge Function so provider credentials remain off the mobile client.

EverCare Core System Flowchart

Actual startup, authentication, feature selection, and authenticated data/integration flow

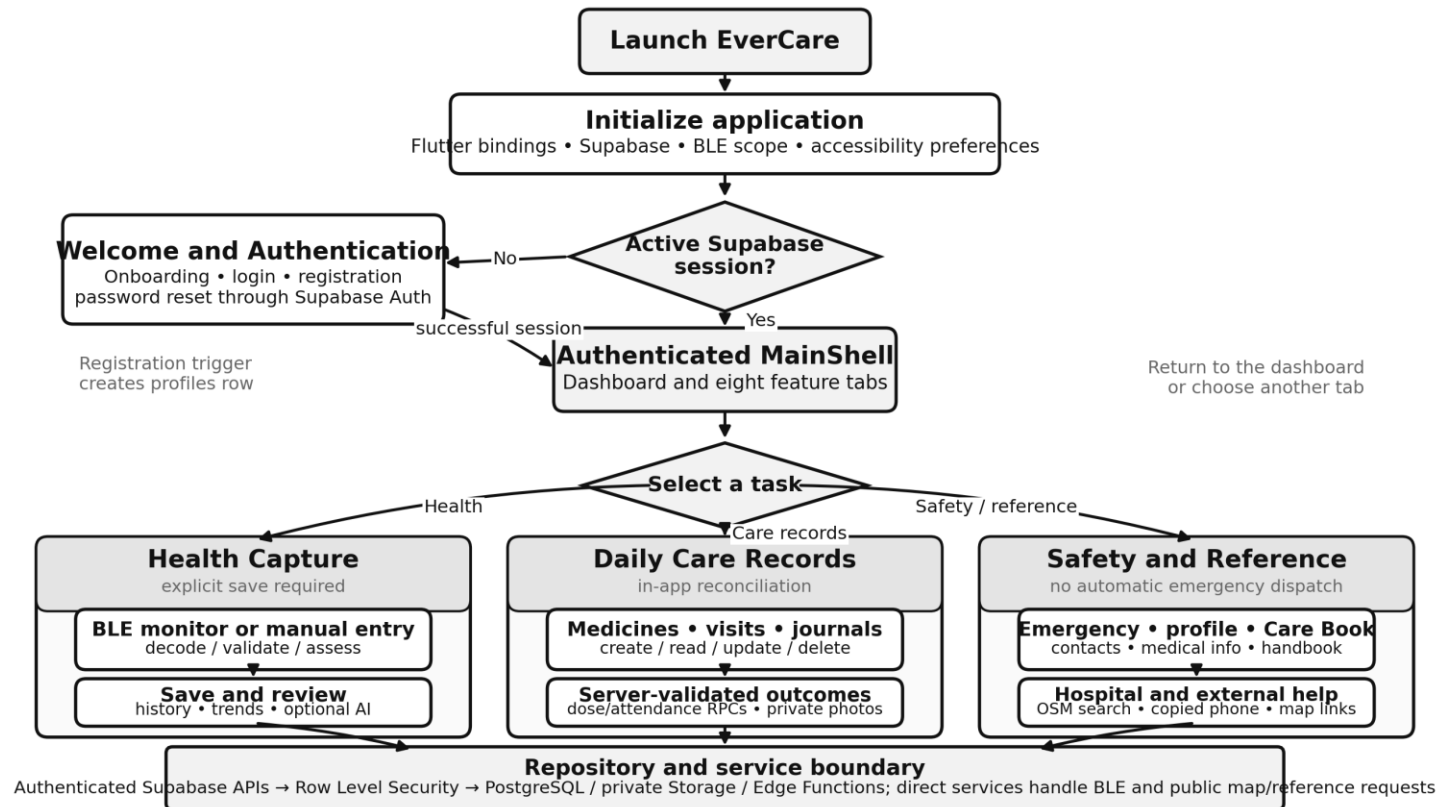


Figure 2. EverCare core system flowchart.

Major Feature Workflows

Blood-Pressure Capture

Health → request Bluetooth permission → scan and save/connect the YK-IBPA1 monitor → discover the custom BLE service and notification characteristic → decode 0x80 progress and 0x81 final packets → apply the implemented −10 mmHg systolic calibration → show the result and assessment → optionally request an AI explanation → explicitly save through BloodPressureRepository. Manual entry bypasses BLE and calibration but uses the same owner-scoped table.

Medication Adherence

Medication screen → load active medicines and recent dose events → calculate Philippine-time occurrences → display upcoming, due, taken, or missed state → call a server-clock RPC for taken/missed → optionally edit, complete, or delete. Missed state is reconciled while the app is active or resumed; there is no background operating-system reminder service.

Appointment Attendance

Appointments → add/edit a visit and optionally select a hospital → persist through AppointmentRepository → open the detail page → use external directions/details or call server RPCs to complete/cancel. Unresolved upcoming visits become missed 24 hours after the start time when reconciliation runs in the app.

Journals and Private Photos

Journal list → create/read/edit/bookmark/delete journal_entries. Camera/gallery photos are resized or compressed, uploaded to the private journal-photos bucket, described by journal_entry_photos metadata, and displayed through temporary signed URLs. Deleting an entry removes its photo objects and metadata through coordinated repository operations.

Emergency and Hospital Finder

Emergency → load the signed-in user's contacts → add/edit/delete and select at most one primary contact → copy a selected number when needed. Hospital Finder requests foreground location, uses OpenStreetMap tiles and public Overpass/Nominatim/Photon services, and can open external Google Maps directions or details. It does not place a call or dispatch emergency assistance.

Care Book, AI, Notifications, and Trusted People

Care Book content and the NIA handbook PDF are bundled with the app. Care Book and blood-pressure AI features invoke authenticated Edge Functions that call Groq; the mobile app does not contain the provider secret. The Notifications screen reads database-backed records and can update is_read, but current code does not generate rows or provide push delivery. Trusted People currently lists recipient-side stored relationships and allows removal; invitation/acceptance UI and access to another user's health records are not implemented.

Database Design Overview

EverCare uses Supabase-hosted PostgreSQL 17. Five versioned migrations define 11 public application tables. Supabase-managed auth.users provides account identity, while storage.objects holds private journal-photo files. Most owner foreign keys point directly to auth.users.id; profiles is not the physical parent of the health and care tables.

- Identity and trusted care: profiles, medical_profiles, and caregiver_relationships.
- Health and scheduling: medications, medication_dose_events, appointments, and blood_pressure_readings.
- Journal and safety: journal_entries, journal_entry_photos, emergency_contacts, and notifications.
- Principal child relationships: medications 1:N dose events; journal entries 1:N photo metadata; profiles participate in caregiver relationships in two roles.
- All 11 public tables have RLS enabled. Storage policies also restrict private photo paths and access to the signed-in journal owner.

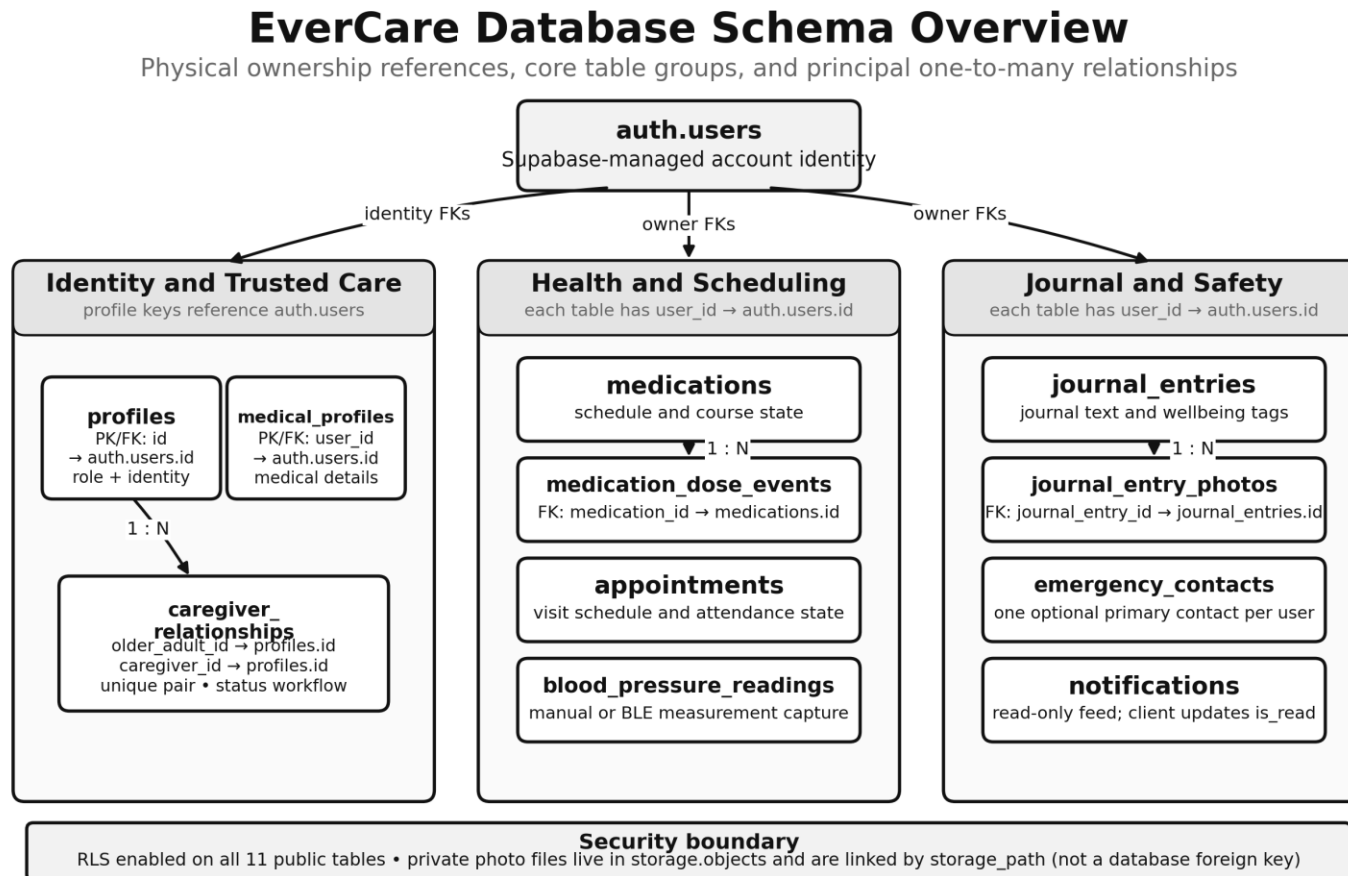


Figure 3. Database schema overview with physical ownership references.

EverCare Database Entity Relationship Diagram (ERD)

PostgreSQL Schema, Primary Keys (PK), Foreign Keys (FK) & Relational Mapping

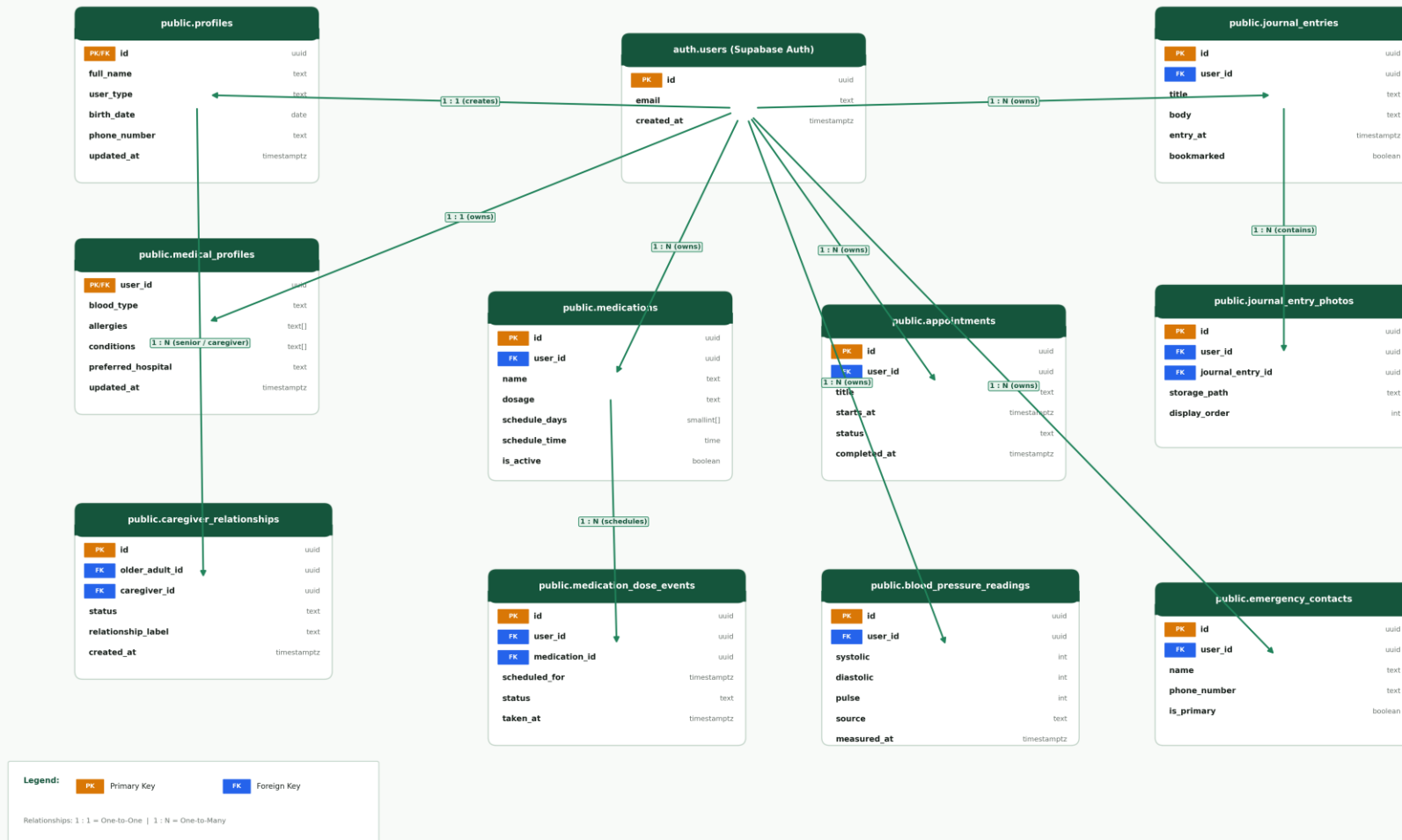


Figure 4. EverCare core entity relationship diagram (ERD).

Scope note: this existing high-resolution ERD emphasizes the core relational feature tables. Figure 3 and the schema dictionary are authoritative for the complete 11-table model, including notifications and the separate auth.users ownership FK on journal photo metadata.

Database Schema Dictionary

The following compact dictionary reflects the final state after all five migrations. A question mark denotes a nullable field; unmarked fields are NOT NULL unless the rule text states otherwise. Supabase-managed auth.users and storage.objects are referenced but are not recreated by the local application migrations.

Identity and Trusted-Care Tables

profiles

Keys and relationships: id uuid is both the PK and an FK to auth.users.id with cascade delete.

Columns: id uuid; full_name text; phone_number text?; birth_date date?; user_type text; address text?; avatar_path text?; created_at timestamptz; updated_at timestamptz.

Rules and indexes: user_type is senior, caregiver, or family_member. A signup trigger inserts the row; updated_at is maintained automatically. Owner can select/update/insert but has no client delete policy.

medical_profiles

Keys and relationships: user_id uuid is both the PK and an FK to auth.users.id with cascade delete and auth.uid() default.

Columns: user_id uuid; blood_type text?; allergies text[]; conditions text[]; preferred_hospital text?; medical_notes text?; created_at timestamptz; updated_at timestamptz.

Rules and indexes: Arrays default to empty. Owner-scoped CRUD is enforced by RLS; updated_at is automatic.

caregiver_relationships

Keys and relationships: id uuid PK; older_adult_id and caregiver_id are FKs to profiles.id with cascade delete.

Columns: id uuid; older_adult_id uuid; caregiver_id uuid; relationship_label text?; status text; created_at timestamptz; updated_at timestamptz.

Rules and indexes: Parties must differ and the pair is unique. Status is pending, accepted, declined, or revoked. A trigger prevents changing parties/label; RLS provides party-specific read/create/update/delete rules.

Health and Scheduling Tables

medications

Keys and relationships: id uuid PK; user_id uuid FK to auth.users.id with cascade delete and auth.uid() default.

Columns: id uuid; user_id uuid; name text; dosage text; purpose text?; frequency text?; instructions text?; schedule_time time?; start_date date?; end_date date?; is_active boolean; schedule_days smallint[]; completed_at timestamptz; created_at timestamptz; updated_at timestamptz.

Rules and indexes: Name/dosage are nonblank; end date cannot precede start date; ISO weekdays are unique values 1–7; completed_at requires inactive state. Indexed by owner, active flag, and time.

medication_dose_events

Keys and relationships: id uuid PK; user_id uuid FK to auth.users.id; medication_id uuid FK to medications.id; both cascade on delete.

Columns: id uuid; user_id uuid; medication_id uuid; scheduled_for timestamptz; taken_at timestamptz?; status text; created_at timestamptz; updated_at timestamptz.

Rules and indexes: Status is scheduled, taken, missed, or skipped. taken requires taken_at; other states require null. medication_id + scheduled_for is unique. Owner/parent consistency is enforced by RLS and RPC checks.

appointments

Keys and relationships: id uuid PK; user_id uuid FK to auth.users.id with cascade delete and auth.uid() default.

Columns: id uuid; user_id uuid; title text; doctor_name text?; specialty text?; starts_at timestampz; clinic text?; address text?; notes text?; status text; completed_at timestampz?; created_at timestampz; updated_at timestampz.

Rules and indexes: Status is upcoming, completed, missed, or cancelled. completed_at is valid only for completed state. Server-clock trigger and RPCs restrict rescheduling and attendance transitions; start-time indexes support owner queries.

blood_pressure_readings

Keys and relationships: id uuid PK; user_id uuid FK to auth.users.id with cascade delete and auth.uid() default.

Columns: id uuid; user_id uuid; systolic integer; diastolic integer; pulse integer; measured_at timestampz; source text; monitor_name text?; decoder_name text?; raw_packet_hex text?; capture_metadata jsonb; notes text?; is_medically_verified boolean; created_at timestampz; updated_at timestampz.

Rules and indexes: Ranges are systolic 1–350, diastolic 1–250, and pulse 1–300; source is ble or manual; verification is constrained to false. A trigger makes capture fields immutable. Indexed by owner/time; BLE packet identity is deduplicated when non-null.

Journal and Safety Tables

journal_entries

Keys and relationships: id uuid PK; user_id uuid FK to auth.users.id with cascade delete and auth.uid() default.

Columns: id uuid; user_id uuid; entry_at timestampz; title text; body text; mood text?; symptoms text[]; activities text[]; tags text[]; bookmarked boolean; created_at timestampz; updated_at timestampz.

Rules and indexes: At least title or body must be nonblank. Arrays default to empty; bookmark defaults false. Owner-scoped CRUD and owner/time index apply.

journal_entry_photos

Keys and relationships: id uuid PK; user_id uuid FK to auth.users.id; journal_entry_id uuid FK to journal_entries.id; both cascade on delete.

Columns: id uuid; user_id uuid; journal_entry_id uuid; storage_path text; display_order integer; created_at timestampz.

Rules and indexes: storage_path is unique and must begin user_id/journal_entry_id/. Display order is nonnegative. RLS confirms both owner and parent; indexes support ordered entry photos and owner history.

emergency_contacts

Keys and relationships: id uuid PK; user_id uuid FK to auth.users.id with cascade delete and auth.uid() default.

Columns: id uuid; user_id uuid; name text; relationship text?; phone_number text; is_primary boolean; created_at timestampz; updated_at timestampz.

Rules and indexes: Name/phone are nonblank. A partial unique index allows at most one primary contact per user; owner-scoped CRUD applies.

notifications

Keys and relationships: id uuid PK; user_id uuid FK to auth.users.id with cascade delete and auth.uid() default.

Columns: id uuid; user_id uuid; title text; body text?; kind text; is_read boolean; created_at timestampz.

Rules and indexes: Title is nonblank. Authenticated clients can select owner records and update only is_read; there is no client insert/delete grant or current in-repository notification producer.

Database Security and Integrity

Row Level Security and Grants

Data area	Effective authenticated-client access
profiles	Owner select, update, and insert; no client delete policy.
medical_profiles, medications, appointments, journal_entries, emergency_contacts, blood_pressure_readings	Owner-scoped select, insert, update, and delete using auth.uid().
medication_dose_events	Owner-scoped CRUD plus a parent-medication ownership check.
journal_entry_photos	Owner-scoped CRUD plus a parent-journal ownership check.
caregiver_relationships	Either party can read/delete; recipient creates pending/revokes; caregiver accepts or declines pending relationships.
notifications	Owner select and update; grants restrict update to is_read. No client insert/delete.
journal-photos Storage	Owner upload/read/delete under a user/journal path; bucket is private and no object update policy exists.

Private Photo Storage

The private journal-photos bucket permits JPEG, PNG, WebP, HEIC, and HEIF objects up to 8 MiB. A journal owner uploads under `user_id/journal_entry_id/...`, and database metadata stores the object path and display order. Reads use a short-lived signed URL. The storage path is a logical link, not a PostgreSQL foreign key.

Server-Side Functions and Triggers

Function or trigger	Purpose
handle_new_user / on_auth_user_created	Creates a profiles row from validated Supabase Auth registration metadata.
get_my_caregiver_relationships	Returns recipient-side relationships and reveals caregiver details only when accepted.
record_medication_dose_taken / missed	Uses the server clock, validates ownership and occurrence age, and writes idempotent dose outcomes.
complete_medication	Marks the owner's course inactive and records a server completion time.
reconcile_missed_appointments	Marks the owner's unresolved visits missed after the 24-hour threshold.
complete_appointment / cancel_appointment	Applies server-validated attendance transitions.
set_updated_at triggers	Refresh updated_at on the nine mutable tables that contain it.
care-party immutability trigger	Prevents changing relationship participants or label after creation.
BP capture immutability trigger	Prevents edits to the stored device/manual measurement capture fields; notes may still be updated.
appointment transition trigger	Rejects invalid schedule/status combinations and assigns trusted completion timestamps.

Implementation Reference

Repository path	Responsibility
lib/main.dart, lib/app.dart	Application initialization, system UI, Supabase initialization, global scopes, lifecycle handling, and MaterialApp.
lib/routes/	Named route constants, route generation, and navigation observation.
lib/screens/	Feature-specific user interfaces and screen-local state.
lib/widgets/	Reusable UI elements plus backend, BLE, and accessibility scopes.
lib/models/	Typed Dart domain and transport records.
lib/repositories/	Owner-filtered data queries, CRUD operations, Storage coordination, and PostgreSQL RPC calls.
lib/services/	Authentication, BLE, hospital lookup, AI, accessibility preferences, and Care Book/download integrations.
lib/decoders/	YK-IBPA1 packet decoding and separation of progress versus completed-result packets.
supabase/migrations/	Authoritative PostgreSQL tables, constraints, indexes, triggers, functions, grants, and RLS policies.
supabase/functions/	Authenticated BP and Care Book Edge Functions and shared Groq integration.
test/	Widget, model, repository, decoder, and service-level automated tests.

Current Scope and Limitations

- Account user_type does not change authorization, navigation, or the feature set; all authenticated roles use the same shell.
- Trusted People is a recipient-side foundation. Invitation/acceptance UI and cross-user health-record sharing are not implemented.
- Medication and appointment reminders are reconciled in-app; no background operating-system notification scheduler or push-notification service is present.
- The notifications table is a read/mark-read feed. Current application and migration code do not produce notification rows.
- Emergency actions copy numbers and open guidance/maps. There is no automatic dial, SMS, SOS, caregiver alert, or emergency-dispatch integration.
- Hospital lookup uses public OpenStreetMap infrastructure intended for low-volume development/classroom demonstration; results and service availability must be verified.
- Saved BP readings are not medically verified. The YK-IBPA1 decoder/calibration is provisional and requires explicit user confirmation before persistence.
- AI output is educational/supportive, not clinical advice. Groq is called only from authenticated Edge Functions, and provider credentials are not stored in the mobile app.
- There is no administrator or clinician portal in the current repository.

Source of Truth

This documentation was prepared from the checked-in EverCare source. The following files and folders should be consulted first when the application changes:

- `supabase/migrations/` — database schema, RLS, functions, triggers, grants, indexes, and private photo policies.
- `lib/main.dart`, `lib/app.dart`, `lib/routes/`, `lib/screens/home/main_shell.dart` — startup, dependency composition, routing, and main navigation.
- `lib/repositories/` and `lib/services/` — implemented data flows and external integrations.
- `README.md` and `docs/AI_SETUP.md` — current setup, operational behavior, and integration limits.
- `docs/week 6/` — editable Mermaid summaries, high-resolution figures, migration verification notes, and demo material.

Conclusion

EverCare combines an accessible Flutter client with an owner-scoped Supabase data layer, private journal media, server-validated care rules, BLE blood-pressure capture, and carefully bounded external integrations. The database schema, RLS policies, triggers, RPCs, and explicit implementation limits documented here define the current system boundary and provide a dependable reference for future development and evaluation.